

IST Interviews Open Coding (sanitized version)

Q: question

A: answer / Info / Problem / Solution / Tool

Introduction Section

XXX: I've been working at XXX for 17 years, so for a while working more or less my whole career out of customer place as a consultant within QA. So yeah, much, very much on the strategic level, putting QA strategy and helping them to implement it on a high level and experience in test automation practices. Yeah, test, I would say tests in general today, test nation is a part of it. And I'm not a developer or it was very long time ago, but there is no, it's more about this the how to put test automation and why you should implement it.

We two years ago have started what we call the tech team at XXX. So more I would say experts within relevant areas where we try to provide with that team more solutions, not one expertise, one guy, it's more a team that provide a solution to address 1 specific problem. Test automation could be one of those, for example. Ok, I can hand it over to Marcus maybe.

My name is XXX. I been working for XXX for almost six years now, but I've been in the industry since second-half of 2000 (ndr, 24 years). I have been working with as a developer, but and test developer and for test automation.

XXX: I'm new since one month at XXX. I have a background in software development and I have been the last 10 years or so a little bit more been working as a data scientist. I've started to to learn a lot about LLMs. And my role here is to both try to implement or try to find a way to implement using LLMs and AI in testing, but also the other way around to add quality assurance to AI systems and machine learning systems.

Hi, my name is XXX, been working at XXX for 11 years and working with test automation in total for 13 or 14 years, both on front end and back end. I'd also been more and more into being coming out some kind of DevOps engineer and in the latest year and a half or something working more with service or tool development, trying to work with some innovation parts regarding how we can use AI and so on. And yeah, maybe one of the most one of us here that's close more most closely working with test automation.

Interview Section

Q: So what type of artificial intelligent have you been using in your work?

A: It has mostly been for instance, and and and Rick mentioned LLMs, but there are many other techniques, right. Yeah, I think at least like generative AI has been used like mostly in the common work at least I mean ChatGPT or CoPilot and and this stuff on a day-to-day basis at least. And I have experience with a lot of different machine learning techniques, all the way from the most basic ones like support vector machines and decision trees, all the way up to computer vision, which I have done a lot in the in the past couple of years. Computer vision, yeah, like object detection and segmentation and image classification.

Q: And can I ask for what kind of problems were you applying computer vision?

A: It's mostly a problem with knowing what's right (ndr, visual oracle) and not like translating your results from from the machine learning model into something that is usable in the real world.

So say that I had an example that I, I bring up quite a lot, which I worked for before I started that XXX where we detected cracks in concrete sleepers for red ways (vision problem). You can use a segmentation model to, to sort of find the cracks in the in the sleepers.

Q: I think you want to just mention it copilot and and ChatGPT.

A: We're running in in our work and analyses, we run quite a lot of interviews (with clients and customers). Gathering information and definitely and writing report and so on (requirement elicitation). So it's more than we, we utilise that to identify that depth and then do some kind of work with the customers.

That's a component of AI that we utilise to identify, we run the tool on the customer's code base and you identify some calls for action. Let's say areas for what regards test planning or the test design or test automation infrastructure, for example.

Q: Do you use it for identifying critical paths in the software or planning what to test, what to, to make what not to automate how that does decisions make you want maybe you're the best one too?

A: Yeah, I mean try to there are some built in technologies and like in in Azure DevOps and so on that they can somehow determine which test cases to run and so based on code changes and etcetera.

But maybe our approach when it comes to automated tests, I would say there is there are three levels, I mean more on close to the developers for unit test level, API and integration level and then GUI level. And what we provide to our customer is more based on their needs. It might be so that it's more relevant for them to start with one area more than another one.

And what, and that approach is very much of what you're saying what, what is critical for them is it, is it a few flows that they want to test automated on the GUI level, even though it's maybe the most expressing part, but they want a few test cases for regression purposes.

Usually we try to guide them on the API level because it's a cheaper way and can get quite a lot of of regression coverage on the API level and maybe do the UI more manually.

Unit test is more is that spot that we, we could, how do you say introduce the concept of unit test because we think it's needed, but that's nothing we provide since usually it's done by the developer themselves. So it's more a mindset from a from a quality very early in the process put on the developer themselves. But after that you have API and UI and there we create what is relevant for the customer.

But it's very much customer driven I would say, since we are here for them. But we of course provide our expertise and guide them in what could be relevant. But this is very much based on their needs. And then we do an approach based on that (customer ad-hoc solutions).

Q: So with respect to developing tests, which is typically a very manual driven kind of activity, how do you integrate AI in that?

A: I will not say that I've used so much in the basis of how to create test cases. It's most, I mostly like use Copilot or or ChatGPT for creating some to create creative solutions on the framework level so that so that the test could stay clean. But then again, we have some AI initiatives where I try to actually use AI to create test cases as well but that's in the very early stages at the moment.

Q: And we have to describe a little bit what we think is relevant with where would you think AI is relevant at the moment?

A: I was going to say, do you where that's your focus, but maybe to require from where we see the value of AI at the moment once again, very we are on in innovation stages. So playing around of course, of course, of course, but it's very exploratory study.

So the part that I've been investigating a lot in is on the API level where you basically in most cases have some kind of documentation or specification of the API in an open API specification or something. And I've tried to use an AI to analyze that specification in some stages.

First I needed to split it up to limit the amount of data that I'm sending to get some valid answers back until I had what I would call like **an endpoint model instead** of the entire API. And then based on that endpoint model started to send different **types of prompts** to see how AI, how AI thinks that I should test this in different matters. Then we came up with something of a **quite static test model** on how AI could describe these test cases in a basic json format (**ndr, creation of API test mockup**).

So building up request bodies, expected responses and so on and even the possibility to say if it could not be sure what to say on one property or, or a status response or whatever, then it could set a property to skip (**brace problem**). And then we add some added some logic in the framework to handle those skipped properties on where we could improve the model in the future (**complement manual testing**).

So that is something that we've tested around a little bit with, but then mostly where, where I see where where we can really benefit from it is of course by **analyzing the test results over time**, because the amount of data, test reports and logs, etcetera can give you.

And even the best thing would be if that analyzing part could even **improve the test cases that you created in the first step** so that you can run this multiple times and improve the the test suite and the test cases over time.

And I can add a little bit to this because I've also started looking a bit, a little bit at this problem. And I'm discussing with XXX quite a lot about how to sort of ensure that the test cases that the **LLMS produce are actually valid and have a good test coverage**. And we're discussing something called **mutation testing** where you, you so say that you, you have your **LLM generated test case and then you mutate the code little bit and then that test test case should fail for that mutated code**. And if it doesn't, then there's something wrong with that test case. And my idea then is to both take the output from the test and and the test code itself and then run it back into the LLM to get sort of an improved test case.

He created a **mutation testing framework for .net** for example, that we used on unit test levels and it and it quite quickly gave good results, or I mean bad results if how you would say.

Q: Don't you have already instances of real faults that you experience? Why do you want to generate so this kind of artificial faults?

A: So, so this is just to ensure that you have good test cases to generate the best test cases. Avenue worth pursuing, but just to ensure that the quality of the test cases that the LLM generates is good enough. And this could also be something that you only run during a development phase when you sort of develop your prompt you. Figure out how much context you need to give to say copilot, what you should send in and also what the sort of the API documentation should look like. So during this process, I think that it could be could be useful. But then maybe if you see that, yes, we get good quality test cases just from the first shot at trying to produce them, then we don't need this mutation testing later.

And I if I can jump in there because if, if we succeed, for example, because first of all, from a general, general point of view, what we're addressing the quality of a test case is something that is very important, but it's quite difficult actually when it's done manually and you have to test designer and they're aware of the application testing are usually they come to that point that they create the test cases that make more value for them.

Q: How do we know that what test cases are relevant?

A: This is very much discussed. Usually when, for example, as a career manager, you said, I want 85 coverage percent of unit tests. You can create that. But the fact is that quite often what you get is you get a developer that create test cases that passes to reach that 85% and instead of the opposite you want test cases actually challenge this.

So I like the approach of that we take responsibly. **We're creating test cases with the help of AI. But on the same time, we use AI to help us to actually validate and and increase quality of what AI just created.**

Q: Is coverage the main metric or do you also have some other quality indicators that you consider important?

A: And that's very philosophical at the moment. But and I worked with a safety quick system and so on where you actually have those define. I'm not usually I'm not saying that to reach a coverage KPI is something very you need to be owning everything that you do that's to make those relevant. Otherwise it's it's very much utilized on compliance level because yeah, we try to quantify quality and that could be through coverage, but usually it's not one to one it needs, you need the quality of what of your coverage as well.

I mean, coverage is a pretty dangerous metric because everyone thinks that if you have high coverage, you have high quality. And I mean, you can have 95% coverage, but all your issues are hidden in those last 5%. The only thing we could say is that 100% is a good metric and of course a higher indicates something may be better than a lower one. And I, and I would also say that if you actually are using the metric and it's, it's at least showing that you have some kind of quality mindset if in indifference to if you don't use it at all. And you should always strive to have a higher coverage.

But a high, a high code coverage or test coverage is not equal to high quality, which is why I think since I'm in a few meetings with, yeah, I don't know what to call them, but yeah, people who are demanding different KPIs and one of them is always coverage.

Yeah, I agree coverage is good if you cover everything otherwise it does might not give much. I was also wanted to add something that yes, 100% coverage is good, but it doesn't really doesn't really mean anything unless you assert your results.

Q: So having the correct assertions should be a better KPI, but it's also difficult because that's would be a manual process to ensure that you have all the assertions that you need?

A: It's more difficult to to add automatically or to measure automatically a better, a better KPI. There are like you mentioned, I make you have a defined setup of test cases on all levels. You need integration and so on and you best KP as a result of those with the approach that every time you have actually a defect in the system that you haven't find, then you create the test cases to cover that problem.

And if you don't do that over time, you will sooner or later have a coverage that is quite high that creates the confidence you need. And then you can use the KPI that is basically the pass rate of your test cases. But you need to reach that one and work with the mindset, but of always creating test cases on everything that fails. So you increase your coverage rate of failure at least of critical failure and that and then you have a relevant KPI. But then you have a team that can rely on exit results of these executed test cases because then they have created reasonable coverage for their needs and for their criticality.

Q: So where do you see like AI kicking in for improving, let's say the test execution process?

A: Another aspect in which I think I mean from our study is being used is like the text execution in terms of infrastructure, in terms of reporting, in terms of identifying the flakiness or process useless test cases that do not need to be re executed rather than paralyze them. Yeah, I mean, as mentioned before, like where I've used a few tools in in Azure DevOps that can give you these kinds of reports of flakiness over time and like long executing test cases. And these kinds of data have not watched worked so much with it in the past recent years.

I would see it being very useful for those teams who are heavy on the UI testing part, since test execution is quite extensive to prioritize your test cases or that if you can map them to requirements or categories and so on. And the same goes to if you, if you look at something like code seeing on how it can prioritize your refactoring.

That could also be good inputs on how much and where you should focus your test cases or the the test suites on I, I can just to add to that, you could do that with sort of statistical methods like Markov Chain Monte Carlo for instance. You could determine where you should, where you should apply your tests and you don't have to sort of run all the tests, but you can select tests to run depending on where in the code they are, how often the code has changed in the past month or so, and also where you have discovered problems in the past.

It's mostly what you get out-of-the-box in Azure DevOps when you're running tests in a pipeline. There over time you get different dashboards and they tag test cases as flaky or long running and and so on. If it's AI behind it, I don't know, but it's a good report that I sometimes use when I'm working with manual testers also to show them what happens over time and so on.

But I also wanted to add something where probably AI could be useful because like in the traditional when, when I've been working in different test projects, you awfully like tag different test cases where this is, this is a sanity test case, this is a smoke test (test case labeling). But then, then you run, you create different types of test pipelines to run dependent on what types of code has been changed in what services and so on. But sometimes it could be quite tricky if you don't have the domain knowledge to, to let's say that you, you change something in service X and then in service said why you have dependencies on, on that, on that service. And I mean, if you don't, if you don't know that you might not run those test cases.

And maybe that is somewhere, some places where AI could help to like find these types of different dependencies in between services to know like, OK, you changed here and now we need to run test cases here. And it's and it's also connected maybe to code that never changes. Then should these test cases be run as often? Maybe they should only be run a few times a year or maybe not at all.

And I sometimes maybe I will get scared of not running it because I think the moment I take that test cases, that test case away, we will have issues on that part of the of the system.

But but maybe, yeah, I don't know some because I mean, mostly when working exploratory with test automation, you try to get as much coverage as you can. You try to write as much test cases as you can and then then try to prioritize based on on the business criteria of that.

And maybe over time, it should be good to have some refactoring regarding all the test cases to OK, maybe you should just have ten of these test cases out of these hundred that you've that you've written during exploratory because now it's legacy and you just need to cover the very basic part of the requirement. So that's kind of input would of course be good as well, but it's not usually performed in because even if I often complain that the developers only focusing on new features and not the regression, that could also be applied to test automation engineers as well.

I will suggest that being paranoid is a very great skill for a tester. Yeah, I'm a pessimistic paranoid person.

Another colleague is to find an anomalies in the test logs, the logs coming out from test cases. And then she's using some kind of different models to analyze this data and get the insights from that. I think she's actually monitoring the systems and taking out the production. Yeah data or stage environment data to do some predictions on issues or exceptions and try to prevent them in before they actually happen.

Q: On my schedule, I have like now a topic which is typically pretty big when it comes to testing, which is test maintenance. And here I think other than the stuff that we discussed is typically the phase in which like companies incur the major cost especially for big system, for big test suites. Also when it comes for instance to debugging to understand whether the problem is in the test or is there an actual fault in the system. So when it comes to maintenance or debugging, what are the main activities that you are doing and how do you AI helping with that, if any?

A: I'm mostly focusing on the framework, parts of the of the test, this test framework and not actually the test. I think I'm always, I'm always thriving to not have to refactor my test as much because then test automation becomes quite expensive over time. But creating test cases is a is the mindset.

When you develop code, you add some things on your daily work that you will need to do forever. I would say when the approach and but with that said, you need to be smart on that part because if you don't do it in a smart way, then you will need to redo test case (TDD). That over time in an efficient way, that's where the expertise needed.

You want a good example? You want roll out the framework to an organization, to a team, I think yeah, perfectly. Instead of taking consultant that we want to provide, we do automated tests. And after a while the automated test case doesn't do what they should do, create incompetence. Providing automated tests where a customer doesn't see that at the problem doesn't address this maintenance strategy (decide what to test), I would say, yeah, yeah, we become automated tests, classical

problem and that's the expertise we want to provide, but it's quite difficult to quantify it for our customers.

Companies also outsource their testing to countries where the developers are typically cheaper. But in this way they create a huge technical depth.

Q: It is really not a good choice because they're bringing their problems on their own and it's crazy on a on a higher level, testing and test maintenance is sort of an accountability problem, right?

A: You, you have different ways to solve accountability within, within software development and one of them is testing, another is code review, a second is pair programming and so on. So that you always have a couple of eyes or, or more ways to look at the code and understand the code and see that it works correctly. I have some hope that instead of maybe having two programmers looking at the same code, you could use AI as an assistant (AI for code review) to help you with that.

And, and that's also something that can help out with this maintenance, maintenance problem that you, you teach your, your developers to. Well, if you change your code, you should also have your AI update the, the test case or ideally do it automatically.

And, and yeah, then instead of just letting the AI write the code, maybe you write the code yourself or you write the test and, and let the AI write the code, something like that.

Q: I think it's very if you run test cases only and don't provide the the results to every relevant stakeholder of the results, then you missing a very important part of those.

A: So I think reporting and how you work with with the results is definitely an important part of the of the framework, I would say initially. So you are able to get the feedback loop as efficient as possible.

So it's not for CI being used for data-driven decision like analyzing the reports and telling you, OK, we should do this tomorrow.

Q: Do you see it like as a as a tool or would you prefer maybe having the report and then you take the final decision?

A: I was not thinking of AI at the moment when I was saying the small process of way of working that provide when you run out to make the test, you should provide all relevant information better the execution to 11 people. When I change the code, some code area here, I quite often get some kind of results and can't see relation between changes and reporting.

This is exactly what I talked about when I described this problem with with computer vision, right? That yes, you could use machine learning or AI to analyse these reports, but you need to be able to understand what you should, what kind of data you should produce and how you should interpret them.

So I think at the first stage, monitoring is the way to go and maybe you could look at trends correlating, for example, code complexity with failed test cases over time, something like that. And, and you don't need AI for that.

But once that you get sort of a more mature understanding of your code base and you have so many KPIs or so many variables that you can look at, then you could start looking at into adding an AI that could perhaps predict. So when I make code changes in this part of the code, how many test errors will I usually produce? So you could get sort of a risk assessment in that case, but that that that means you would have to have a very mature code base where you have a lot of historical data which are collected during over a long time and then also need to understand that.

So what I'm trying to say is that that AI and machine learning is not a tool that you can just apply here. You need to have an understanding of how to go from A to B and how to interpret your results first.

And then when you can do that, you can sort of minimise the analysis effort by just having an AI machine learning model say that OK, I have looked at these parameters and I predict this result and then you can act on that instead of doing hours of analysis work. But I think we're quite far away from that yet in sort of all the software industries.

But I think that as a whole, software development is quite a bit way off. Yeah, in a way, like in testing, we want to have clear answers and clear responses. And there are like, also phenomena, for instance, when you use LLMs, known as hallucinations. Should we trust this response?

Q: So do you think more hallucinations or say kind of defects are something to control or ladder to leverage for testing because it might also input some creative productivity in your process?

A: Yeah, hallucination might be something that you want. Because this is what we are doing when we do testing, we try to produce unusual inputs for our system.

I don't think that hallucinations are good overall because I don't think it's true creativity. I think it's sort of false creativity. It's yeah, it's it's guessing at something that it doesn't know anything about. And that's, I would say, almost never good.

Q: AI like a fancy term for some more advanced or more intelligent test automation? Or do you see like as something that it is here to stay and it will transform the way we work?

I think that LLMs will transform the way that software developers work, but I also see quite a lot of problems with that. So the, the basis for this must be that this is a tool for every tester or every developer. To be able for junior developers who are not as familiar with testing or familiar with software development, they need some guardrails. They either need some more senior colleague to look at the code that has been produced or they need some way to sort of ask the the model is this really correct so that the model can self check what it has produced, something like that.

But we need to develop best practices on how to use them and we need to especially educate junior developers and testers on how to use them and which pitfalls to look out for.

Yeah, my impression is that AI will empower the skilled developers and we will very detrimental for the unskilled personnel or junior personnel because they might tend to trust just the outcome as it is.

It's just that developers will maybe spend more time reviewing and using tools to review code rather than writing code themselves manually. But you'll still be like the man, the human expertise at the moment.